

Documentación del software fibonacci_tmm

Emanuel Orozco Gallego

31 de agosto de 2026

Resumen

Manual independiente de la biblioteca Python que reproduce Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010), con el método de matriz de transferencia.

Índice

1. Arquitectura	2
2. Instalación	2
3. Uso	2
4. API principal	2
5. Modelo físico	3
6. TMM	3
7. Fibonacci	3
8. Dispersión	3
9. Tests	3
10.Reproducción de figuras	3
11.Solución de problemas	3
12.Extensiones	4

1. Arquitectura

El paquete vive en `src/fibonacci_tmm/`.

Módulo	Responsabilidad
<code>fibonacci.py</code>	S_m , F_m , L_m , inflación
<code>materials.py</code>	Aire y Drude
<code>electromagnetics.py</code>	n , q , Q , TE/TM
<code>transfer_matrix.py</code>	M_j , T_m , R_m
<code>dispersion.py</code>	$\cos(kL) = R$, bandas
<code>plasmon_modes.py</code>	recuento de subbandas
<code>bandwidth.py</code>	intervalos $\nu_{\min}(\theta)$ – $\nu_{\max}(\theta)$ (bordes refinados)
<code>params.py</code>	YAML $\rightarrow$ SI
<code>plotting.py</code>	estilo de figuras
<code>provenance.py</code>	metadatos de corrida

Los parámetros científicos no están en los scripts: están en `configs/figure_0N.yaml`.

2. Instalación

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Python ≥ 3.10 . Dependencias: NumPy, SciPy, Matplotlib, PyYAML, pytest.

3. Uso

```
.venv/bin/pytest
.venv/bin/python scripts/tmm/figure_01.py # una figura, TMM
.venv/bin/python scripts/tmm/run_all.py # Figs. 1-6, TMM
.venv/bin/python scripts/pwe/run_all.py # Figs. 1-6, ondas planas
.venv/bin/python scripts/comparison/run_all.py # TMM vs PWE
make paper pwe-docs
```

Para un orden nuevo de Fibonacci, cambie `fibonacci_orders` en el YAML. Para TM, ponga `polarization: TM`. Para otro Drude, edite `omega_e_over_2pi_ghz` y `omega_m_over_2pi_ghz`.

4. API principal

```
sequence(m), paper_fibonacci(m), build_cell(m,a,b).
scan_dispersion(spec, m, theta, nu_ghz, polarization) (TMM; alias de
scan_dispersion_tmm).
scan_dispersion_pwe(spec, m, theta, nu_ghz, polarization,
harmonics_per_layer=16): la misma semitraza con ondas planas; devuelve el mismo
DispersionScan.
```

`compare_scans(ref, cand)`, `locate_bands`, `validate_bands`: comparación y localización de subbandas.

El paquete es `fibonacci_photonics` (subpaquetes `core`, `tmm`, `pwe`, `analysis`); `fibonacci_tmm` sigue disponible como alias.

```
semitrace_by_recurrence y semitrace_by_product deben coincidir.
detect_plasmon_modes(scan, nu_min_ghz=..., nu_max_ghz=...).
load_spec(path) lee un YAML.
```

5. Modelo físico

Aire $\varepsilon = \mu = 1$. Drude $\varepsilon = 1 - \omega_e^2/\omega^2$, $\mu = 1 - \omega_m^2/\omega^2$. Frecuencia graficada: $\nu = \omega/2\pi$ en GHz.

6. TMM

Matriz de capa unimodular con $\chi = \mu$ (TE) o ε (TM). Producto $T = M_N \cdots M_1$. Recurrencia $R_m = 2R_{m-1}R_{m-2} - R_{m-3}$. $q = (\omega/c)n_A \sin \theta$ (el paper omite ω/c).

7. Fibonacci

$$S_0 = B, S_1 = A, S_m = S_{m-1}S_{m-2}, F_0 = F_1 = 1, N_B = F_{m-2}, L_m = F_{m-1}a + F_{m-2}b.$$

8. Dispersión

Banda si $|R| \leq 1 + 10^{-12}$. $kL_m/\pi = \arccos(R)/\pi$. Los puntos con $|R| > 1$ se dejan como NaN, no se recortan en silencio.

9. Tests

- `core/test_fibonacci.py`: secuencias, inflación, N_B , L_m .
- `core/test_materials.py`: ceros de Drude, ν_e, ν_m .
- `core/test_params.py`: YAML de las figuras.
- `tmm/test_transfer_matrix.py`: $\det T = 1$, (7)–(9), recurrencia vs producto, TE=TM a $\theta = 0$.
- `tmm/test_dispersion.py`: máscara de banda, gap $\langle n \rangle = 0$.
- `pwe/test_fourier.py`, `pwe/test_solver.py`: Fourier de la celda, benchmark del libro, $R_{\text{PWE}} \approx R_{\text{TMM}}$.
- `analysis/test_validation.py`: $N_{\text{modos}} = F_{m-2}$.
- `analysis/test_comparison.py`: métricas y localización de bordes.
- `compat/test_legacy_imports.py`: alias `fibonacci_tmm`.

10. Reproducción de figuras

Cada `results/tmm/figure_0N/` contiene `data/`, `output/` y `README.md` (`figures/` es un enlace a `results/tmm/`). Las versiones PWE están en `results/pwe/figure_0N/` y la comparación en `results/comparison/`. Las imágenes finales son PDF, PNG (300 dpi) y SVG.

11. Solución de problemas

- `ZeroDivisionError` en $\omega = 0$: el Drude diverge; no incluya el origen en el barrido.
- $\mu_B = 0$ o $\varepsilon_B = 0$: polo; el escáner enmascara una vecindad relativa 10^{-6} .
- $|R|$ siempre ≤ 1 a $\theta = 0$ si $\omega_e = \omega_m$: no es un bug, es adaptación de impedancia.

- Fig. 6: el original grafica $\nu(\theta)$ relleno, no $\Delta\nu$; el caption del PRB está incompleto (parámetros inferidos de las Figs. 3–5).
- ImportError: añade `src` al `PYTHONPATH` o use los scripts de `scripts/`.

12. Extensiones

Nuevo orden: `YAML`. Nueva polarización: `Polarization.TM`. Nuevo material: implemente `epsilon_mu(omega)` con la misma interfaz que `DrudeMetamaterial`.